

Proyecto Final

Web Scraping

Edith Lilian Castillo Ojeda
Xiadani Paola Reyes Cuellar

Noviembre 2025

Objetivo

El objetivo del proyecto es realizar una comparativa de todos los audífonos disponibles en la plataforma de **Liverpool** y analizar su comportamiento de precios durante épocas de descuento, como el Black Friday, con la finalidad es verificar si los productos realmente ofrecen rebajas o si los precios son inflados previamente para simular descuentos.

Para ello, se extraerá información mediante técnicas de web scraping utilizando Python como lenguaje de programación.

Pasos a seguir

- **Paqueterías**

Para poder correr el programa primero que nada se necesitaron intalar varias paqueterías,

```
!pip install pandas
!pip install openpyxl
!pip install selenium
```

- **Importaciones**

Posteriormente llamamos las paqueterías que estaríamos utilizando para la ejecución del código

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import pandas as pd
import time
```

- **Web Scraping**

Al ser el propósito principal de este programa realizar **web scraping** en la página de Liverpool, para poder recopilar toda la información de productos relacionados con audífonos, manos libres y auriculares, se creó este programa que navega automáticamente por múltiples páginas, donde extrae todos los datos importantes de cada producto, además se limpia la información, para que finalmente se guarde en un archivo Excel.

Para ello, se creo la siguiente función que sirve para obtener el precio de un producto desde la página web y convertirlo en un formato limpio y fácil de leer.

```
def precio_a_texto(producto, precio_selector="numbersnumbers.
numbersanumbers-numberscardnumbers-numberspricenumbers"):
```

Sirve para obtener un precio desde el HTML del producto y devolverlo formateado correctamente.

Dentro de la función encontramos:

```
precio_elem = producto.find_element(By.CSS_SELECTOR, precio_selector
)
```

Aquí el programa localiza dentro de la tarjeta de producto el elemento que contiene el precio.

Posteriormente se obtiene la parte principal del precio (los pesos), donde toma solamente la primera línea del texto del precio.

```
precio\_main = precio\_elem.text.split(numbers"numbers\\numberstextbackslash  
numbers numbersnnumbers") [0]
```

Limpiamos el texto del precio, donde se eliminan símbolos como el signo de pesos y las comas.

```
precio\_main = precio\_main.replace(numbers"numbers\\$numbers", numbers"  
numbers").replace(numbers"numbers, numbers", numbers"numbers").strip()
```

Continuamos buscando los centavos:

```
centavos\_elem = precio\_elem.find\_element(By.TAG\_NAME, numbers"  
numberssupnumbers")
```

En caso de que falle, el bloque `except` asigna "00".

Convertimos los pesos y centavos a número real.

```
precio\_float = int(precio\_main) + int(centavos)}  
texttt{precio\_float = precio\_float / 100.0}
```

Por último, se formatea el precio

```
precio\_texto = numbers"numbers\\$numbers{:, .2numbersfnumbers}numbers".format(  
precio\_float)
```

En caso de que ocurra un error en cualquier parte, el código devuelve `None`.

Inicio de Driver

En esta sección se abre el navegador Chrome, donde se establece un tiempo de espera máximo de 20 segundos para que carguen los elementos. Y se crea la lista `data` donde se guardarán todos los productos.

```
driver = webdriver.Chrome()  
wait = WebDriverWait(driver, 20)  
data = []
```

Lista de Categorías

Se crea la lista de categorías:

```
categorias = [ {numbers"numbersnombrenumbers": numbers"numbersAud fonos  
numbers", numbers"numbersbusquedanumbers": numbers"numbersaudifonosnumbers",  
numbers"numberspaginasnumbers": 42}, {numbers"numbersnombrenumbers": numbers"  
numbersManosnumbers numberslibresnumbers", numbers"numbersbusquedanumbers":  
numbers"numbersmanosnumbers numberslibresnumbers", numbers"numberspaginas  
numbers": 6}, {numbers"numbersnombrenumbers": numbers"numbersAuriculares  
numbers", numbers"numbersbusquedanumbers": numbers"numbersauricularesnumbers"  
, numbers"numberspaginasnumbers": 33} ]
```

Scraping

Se controla todo el proceso general del scraping en el siguiente ciclo:

```
for categoria in categorias:  
    for page in range(1, categoria[numbers"numberspaginasnumbers"] + 1):
```

```
    for categoria in categorias:  
        for page in range(1, categoria["paginas"] + 1):
```

Se construye la URL:

```
url = fnumbers"numbershttpsnumbers://numberswwwnumbers.numbersliverpoolnumbers.  
numberscomnumbers.numbersmx/numberstienda/numberspagenumbers-\\{numberspage  
numbers}\\?numberssnumbers=\\{numberscategorianumbers[numbers"numbersbusqueda  
numbers']\\}numbers"
```

Se apertura la página:

```
driver.get(url)
```

Se espera a que carguen los productos:

```
wait.until(EC.presence_of_all_elements_located(...))
```

Extracción de Información de cada Producto

Donde este ciclo es controlado por:

```
products = driver.find_elements(...)  
for p in products:
```

Y en cada producto obtenemos: la marca, la descripción, el precio, la URL del producto, la imagen

```
marca = p.find_element(By.CSS_SELECTOR, numbers"numbersh3numbers.numbersa  
numbers-numberscardnumbers-numbersbrandnumbers").text  
description = p.find_element(By.CSS_SELECTOR, numbers"numbersh3numbers.  
numbersanumbers-numberscardnumbers numbersdescriptionnumbers").text  
description = p.find_element(By.CSS_SELECTOR, numbers"numberspnumbers.  
numbersanumbers-numberscardnumbers-numbersdescriptionnumbers").text  
precio_sin_desc = precio_a_texto(p, numbers"numberspnumbers.numbersa  
numbers-numberscardnumbers-numberspricenumbers")  
precio_desc = precio_a_texto(p, numbers"numberspnumbers.numbersanumbers-  
numberscardnumbers-numbersdiscountnumbers")  
url_producto = p.find_element(By.TAG_NAME, numbers"numbersanumbers").get_  
_attribute(numbers"numbershrefnumbers")  
url_img = p.find_element(By.CSS_SELECTOR, numbers"numbersimgnumbers").get_  
_attribute(numbers"numberssrcnumbers")
```

Y por último se gusrada en la lista data

```
data.append(\{ ... \})
```

Cierre del Navegador

Aquí se cierra el navegador y se convierte toda la información recopilada en una tabla.

```
driver.quit()  
df = pd.DataFrame(data)
```

Eliminación de Duplicados

Se eliminan productos repetidos basándose en su URL.

```
df.drop_duplicates(subset=numbers"numbersURLnumbers numbersProductonumbers",  
inplace=True)
```

Filtro

El siguiente filtro nos nos ayuda a aceptar descripciones que incluyen palabras relacionadas con audífonos y rechaza descripciones que incluyan palabras prohibidas como “bocina”, “mouse”, “cargador”, “funda”, etc.

```
def es_valido(texto):  
df = df[df[numbers"numbersDescripci nnumbers"].apply(es_valido)]
```

Guardado en Excel

Se guarda en un archivo Excel con todos los productos ya filtrados y formateados.

```
df.to_excel(numbers"numbersaudifonosnumbers\_numbersynumbers\_numbersmanos  
numbers\_numberslibresnumbers\_numbersliverpoolnumbers\_numbersVentaNocturna  
numbers.numbersxlsxnumbers", index=False)
```

- Cargar archivos

Una vez obtenidos los datos, de diferentes periodos de promoción. Llamamos los documentos exportados en fromato Excel.

```
# =====  
# CARGAR ARCHIVO  
# =====  
# ---Venta Nocturna ---  
# 07 de Diciembre de 2025
```

```

archivo = numbers"numbersaudifonos_numbersy_numbersmanos_numberslibres_
numbersliverpool_numbersVentaNocturnanumbers.numbersxlsxnumbers"
df = pd.read_excel(archivo)

# ---BlackFriday ----
# 28 de Noviembre de 2025
archivo2 = numbers"numbersaudifonos_numbersy_numbersmanos_numberslibres_
numbersliverpool_numbersblackfridaynumbers.numbersxlsxnumbers"
df2 = pd.read_excel(archivo2)

# ---Normal ----
# 20 de Noviembre de 2025
archivo3 = numbers"numbersaudifonos_numbersy_numbersmanos_numberslibres_
numbersliverpool_numberslimpiobuenonumbers numbers(1)numbers numbers(1)
numbers.numbersxlsxnumbers"
df3 = pd.read_excel(archivo3)

```

• Limpieza de datos

Dentro de la siguiente sección, se busca limpiar los datos para poder trabajar con ellos de una manera más optima, ya que convierte toda la columna a texto (string). Esto con la finalidad de que los precios pueden venir como números, texto, NaN, etc.

```

# =====
# LIMPIEZA DE PRECIOS
# =====
def limpiar_precio(col):
    col = col.astype(str)
    col = col.str.replace(numbers"numbers,numbers",numbers"numbers", regex=
        False)
    col = col.replace(numbers"numbersnannumbers", None) #Elimina las comas,
        t picas en los precios.

    return col.astype(float)

df[numbers"numbersPrecionumbers numbersssinnumbers numbersdescuentonumbers"] =
    limpiar_precio(df[numbers"numbersPrecionumbers numbersssinnumbers
        numbersdescuentonumbers"])
df[numbers"numbersPrecionumbers numbersconnumbers numbersdescuentonumbers"] =
    limpiar_precio(df[numbers"numbersPrecionumbers numbersconnumbers
        numbersdescuentonumbers"])

# =====
# ELIMINAR SIN MARCA
# =====
df = df.dropna(subset=[numbers"numbersMarcanumbers"])

```

Por último, eliminamos todos aquellos datos nulos que no contengan un valor:

```

# =====
# LIMPIAR DATOS
# =====

#Eliminamos todos aquellos datos que tengan datos nulos en el precio

df = df[pd.to_numeric(df[numbers'numbersPrecionumbers numbersssinnumbers
        numbersdescuentonumbers'], errors=numbers'numberscoercenumbers').notnull()]

```

Y el proceso lo repetimos con los 3 archivos.

• Comparación de precios

Comenzamos a preparar los datos a utilizar para poder generar el análisis sobre los costos en los diferentes días de promoción y saber si realmente se está haciendo un descuento. Para ello, se realiza una comparación de precios, donde elaboraremos un nuevo documento donde realizaremos la comparación:

```

# COMPARACION DE PRECIOS
Antes = df3

```

```

Black = df2
Nocturna = df

#Renombramos columnas
Antes = Antes.rename(columns={numbers"numbersPrecionumbers numbersconnumbers
    numbersdescuentoantesnumbers": numbers"numbersAntesnumbers"})
Black = Black.rename(columns={numbers"numbersPrecionumbers numbersconnumbers
    numbersdescuentonumbers": numbers"numbersBlackFridaynumbers"})
Nocturna = Nocturna.rename(columns={numbers"numbersPrecionumbers numberscon
    numbers numbersdescuentonumbers": numbers"numbersNocturnanumbers"})

# Cambia este nombre si tu columna se llama distinto
clav = numbers"numbersURLnumbers numbersProductonumbers"
clav1=numbers"numbersMarcanumbers"

```

Eliminamos todos aquellos datos duplicados y logremos trabajar con productos únicos:

```

# ELIMINAR DUPLICADOS
Antes = Antes.drop_duplicates(subset=[clav])
Black = Black.drop_duplicates(subset=[clav])
Nocturna = Nocturna.drop_duplicates(subset=[clav])

```

Y unimos estas columnas en un solo formato:

```

#Unimos
Union1 = pd.merge(
    Antes[[clav, numbers"numbersAntesnumbers"]],
    Black[[clav, numbers"numbersBlackFridaynumbers"]],
    on=clav,
    how=numbers"numbersouternumbers"
)

# Unir el resultado con Nocturna
Union = pd.merge(
    Union1,
    Nocturna[[clav, numbers"numbersNocturnanumbers"]],
    on=clav,
    how=numbers"numbersouternumbers"
)

```

• Clasificación

Ahora clasificamos cada fila como "Disponible" o "Agotado" dependiendo de si ciertas columnas tienen datos o están vacías (NaN), ya que se marcará como "Agotado" si la columna "BlackFriday" o "Nocturna" tiene valores faltantes (NaN), de tal forma que el resultado final sea:

- Estado = "Disponible" → solo si ambas columnas (BlackFriday y Nocturna) tienen un valor.
- Estado = "Agotado" → si al menos una está vacía.

```

# =====
# AGOTADOS
# =====
Union[numbers"numbersEstadonumbers"] = numbers"numbersDisponiblenumbers"
Union.loc[Union[numbers"numbersBlackFridaynumbers"].isna(), numbers"
    numbersEstadonumbers"] = numbers"numbersAgotadonumbers"
Union.loc[Union[numbers"numbersNocturnanumbers"].isna(), numbers"numbersEstado
    numbers"] = numbers"numbersAgotadonumbers"
print(Union)

```

• Comparación

Realizamos la comparación de precios, de tal manera que podamos ver de cuanto fue el descuento durante el BlackFriday y la Venta Nocturna, además de agregar el valor de dicho descuento en porcentaje para ver porcentualmente de cuanto fue el descuento por producto:

```

# =====
# COMPARACION
# =====

```

```
# Vemos si hubo algun descuento durante la venta
Union[numbers"numbersBlackFriday_numbersDifnumbers"] = Union[numbers"
    numbersAntesnumbers"] - Union[numbers"numbersBlackFridaynumbers"]
Union[numbers"numbersNocturna_numbersDifnumbers"] = Union[numbers"numbersAntes
    numbers"] - Union[numbers"numbersNocturnanumbers"]

Union[numbers"numbersBlack_numbers%numbers"] = (Union[numbers"
    numbersBlackFriday_numbersDifnumbers"] / Union[numbers"numbersAntesnumbers"
]) * 100
Union[numbers"numbersNocturna_numbers%numbers"] = (Union[numbers"
    numbersNocturna_numbersDifnumbers"] / Union[numbers"numbersAntesnumbers"]) *
    100
```

- **Clasificación de Descuentos** Con la comparación previamente realiza clasificamos los resultados obtenidos mediante dicha comparación, tomando en cuenta el porcentaje de descuento por cada producto, de tal forma que los resultados obtenidos serían:

- Resultado $> 0 \rightarrow$ *Descuentoreal*
- Resultado $< 0 \rightarrow$ *Subióprecio*
- Resultado $= 0 \rightarrow$ *Sincambio*
- NaN $\rightarrow$ *Agotado*

```
# =====
# CLASIFICAR LOS DESCUENTOS
# =====
def clasificar(x):
    if pd.isna(x):
        return numbers"numbersAgotadonumbers"
    elif x > 5:
        return numbers"numbersDescuentonumbers numbersrealnumbers"
    elif x < -5:
        return numbers"numbersSubi numbers numbersprecionumbers"
    else:
        return numbers"numbersSinnumbers numberscambionumbers"

Union[numbers"numbersResultadonumbers"] = Union[numbers"numbersBlack_numbers%
    numbers"].apply(clasificar)
Union[numbers"numbersResultado2numbers"] = Union[numbers"numbersNocturna_
    numbers%numbers"].apply(clasificar)
Union.to_excel(numbers"numbersComparacionesnumbers.numbersxlsxnumbers", index=
    False)
#print(Union)
```

- **Gráficas**

Representamos en un grafica todos aquellos productos que presentaron un descuento real en el BlackFriday y en la venta nocturna:

```
com = Union[Union[numbers"numbersResultadonumbers"]== numbers"numbersDescuento
    numbers numbersrealnumbers"]
com[numbers"numbersIDnumbers"] = range(1, len(com) +1)

plt.figure()
plt.barh(com[numbers"numbersIDnumbers"], com[numbers"numbersBlack_numbers%
    numbers"], color = numbers"numbersmagentanumbers")
plt.title(numbers"numbersProductosnumbers numbersconnumbers numbersDESCUENTO
    numbers numbersREALnumbers numbersennumbers numbersselnumbers
    numbersBlackFridaynumbers")
plt.xlabel(numbers"numbers%numbers numbersdenumbers numbersdescuentonumbers")
plt.ylabel(numbers"numbersIDnumbers numbersdelnumbers numbersproductonumbers")
plt.tight_layout()
plt.grid()
plt.show()
```

De igual manera representamos todos aquellos productos que aumentaron su precio durante el BlackFriday y la Venta Nocturna:

```
com = Union[Union[numbers"numbersResultadonumbers"]== numbers"numbersSubi
            numbers numbersprecionumbers"]
com[numbers"numbersIDnumbers"] = range(1, len(com) +1)

plt.figure()
plt.barh(com[numbers"numbersIDnumbers"], com[numbers"numbersBlack_numbers%
        numbers"], color = numbers"numbersmagentanumbers")
plt.title(numbers"numbersProductosnumbers numberssquenumbers numbersAUMENTARON
        numbers numbersSUnumbers numbersPRECIONumbers numbersREALnumbers numbersen
        numbers numbersselnumbers numbersBlackFridaynumbers")
plt.xlabel(numbers"numbers%numbers numbersdenumbers numbersdescuentonumbers")
plt.ylabel(numbers"numbersIDnumbers numbersdelnumbers numbersproductionumbers")
plt.tight_layout()
plt.grid()
plt.show()
```

Mostramos cuales fueron los productos que tuvieron un aumento arriba del 50% tanto en el BlackFriday y en la Venta Nocturna:

```
# Marcas que inflaron m s sus precios
marc = com[com[numbers'numbersBlack_numbers%numbers'] < -60]
print(len(marc))
marc

completo = com.merge(df2[[numbers'numbersURLnumbers numbersProductionumbers',
        numbers'numbersMarcanumbers']], on=numbers'numbersURLnumbers numbersProducto
        numbers', how=numbers'numbersleftnumbers')
marc = completo[completo[numbers'numbersBlack_numbers%numbers'] < -60]
print(marc[[numbers'numbersURLnumbers numbersProductionumbers', numbers'
        numbersMarcanumbers', numbers'numbersBlack_numbers%numbers']])
```

Otra comparación fue ver los precios sin descuento contra los precios con descuento durante el BlackFriday y la Venta Nocturna:

```
plt.figure(figsize=(10, 5))
plt.plot(df2[numbers'numbersPrecionumbers numbersssinnumbers numbersdescuento
        numbers'], label=numbers"numbersPreciosnumbers numbersssinnumbers
        numbersdescuentonumbers", linestyle=numbers'numbers-numbers', c=numbers"
        numbersorangenumbers")
plt.plot(df2[numbers'numbersPrecionumbers numberssconnumbers numbersdescuento
        numbers'], label=numbers"numbersPreciosnumbers numberssconnumbers
        numbersdescuentonumbers", linestyle=numbers'numbers-numbers', c=numbers"
        numbersgreynumbers")
plt.xlabel(numbers'numbersArt culosnumbers')
plt.ylabel(numbers'numbersPreciosnumbers')
plt.title(numbers'numbersBlackFridaynumbers numberssennumbers numbersLiverpool
        numbers')
plt.legend() # <-- Muestra la leyenda
plt.grid(True)
plt.show()
```

- **Agrupar y crear un resumen por marca**

Agrupamos los productos por la columna “Marca” y calculamos cuántos productos tiene cada marca, el precio promedio sin descuento y el precio promedio con descuento:

```
# =====
# RESUMEN POR MARCA
# =====
resumen1 = df2.groupby(numbers"numbersMarcanumbers").agg(
        Productos=(numbers"numbersMarcanumbers", numbers"numberscountnumbers"),
        Sin_desc=(numbers"numbersPrecionumbers numbersssinnumbers numbersdescuento
                numbers", numbers"numbersmeannumbers"),
        Con_desc=(numbers"numbersPrecionumbers numberssconnumbers numbersdescuento
                numbers", numbers"numbersmeannumbers")
).reset_index()
```

Calculamos algunas métricas para cada marca, como su diferencia como el cambio en porcentaje, además clasificamos si hubo descuento, aumento el precio o no se presentó ningún descuento. Posteriormente ordenamos las marcas por las 15 marcas con precio promedio más bajo en Black Friday y Venta Nocturna, como las del precio más alto:

```
# M TRICAS
resumen1[numbers"numbersDiferencianumbers"] = resumen1[numbers"numbersSin_
numbersdescnumbers"] - resumen1[numbers"numbersCon_numbersdescnumbers"]
resumen1[numbers"numbersCambio_numbers%numbers"] = (resumen1[numbers"
numbersDiferencianumbers"] / resumen1[numbers"numbersSin_numbersdescnumbers"
]) * 100

# CLASIFICAR DESCUENTOS
def clasificar(x):
    if x > 5:
        return numbers"numbersDescuentonumbers numbersrealnumbers numbers"
    elif x < -5:
        return numbers"numbersDescuentonumbers numbersfalsounumbers numbers"
    else:
        return numbers"numbersSinnumbers numberscambionumbers numbers"

resumen1[numbers"numbersTiponumbers numbersdescuentonumbers"] = resumen1[
numbers"numbersCambio_numbers%numbers"].apply(clasificar)

resumen1[numbers"numbersTiponumbers numbersdescuentonumbers"] = resumen1[
numbers"numbersCambio_numbers%numbers"].apply(clasificar)

# ORDENAR POR PRECIO
top_baratas1 = resumen1.sort_values(numbers"numbersCon_numbersdescnumbers").
head(15)
top_caras1= resumen1.sort_values(numbers"numbersCon_numbersdescnumbers",
ascending=False).head(15)
```

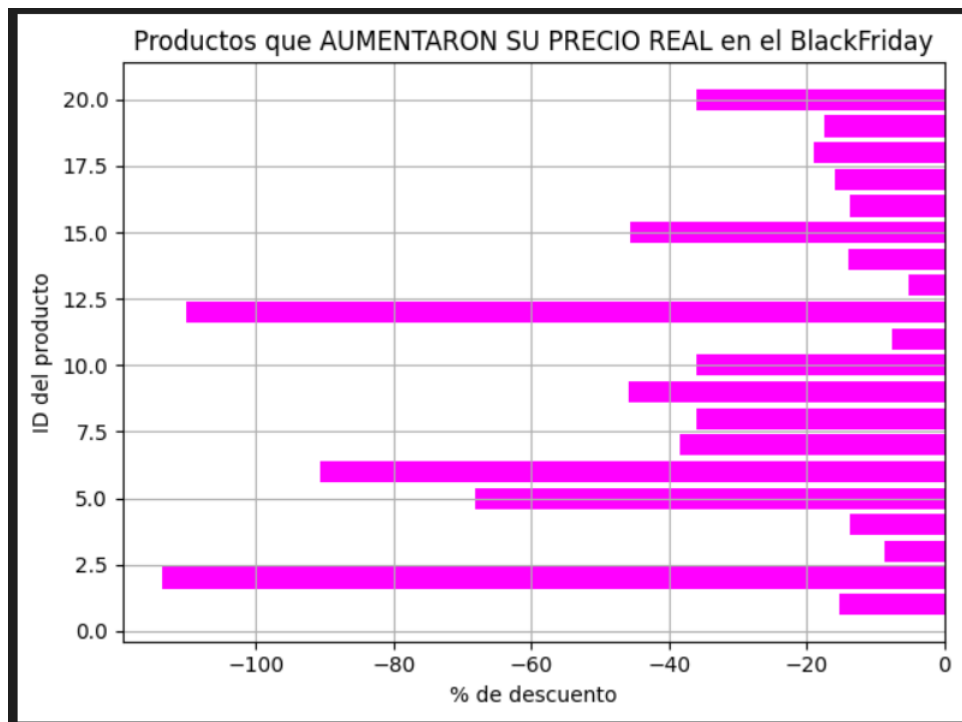
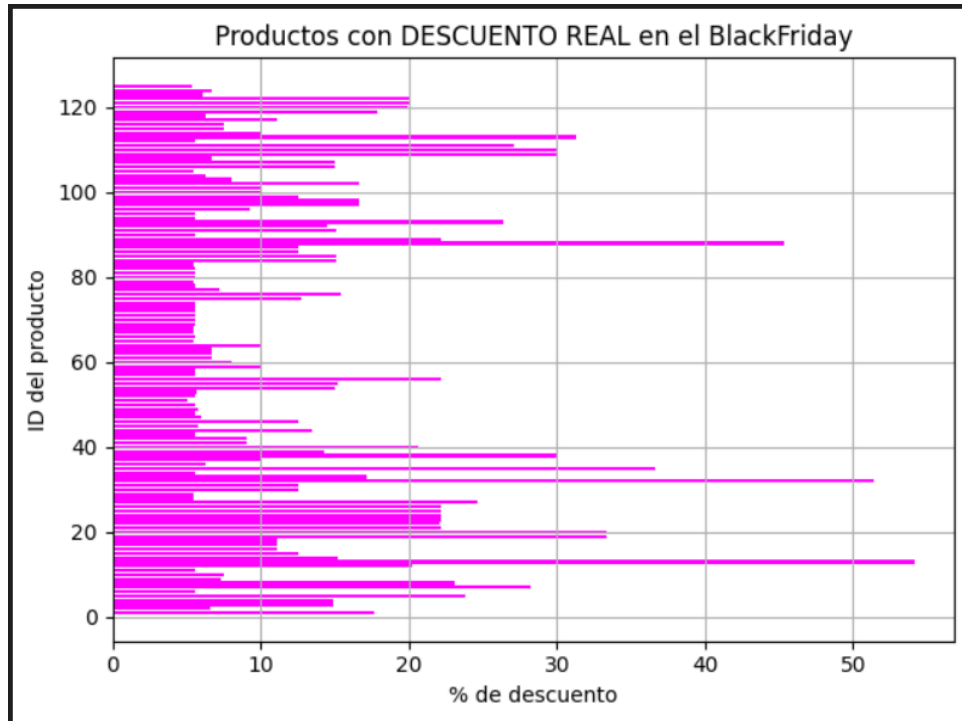
Y por último graficamos:

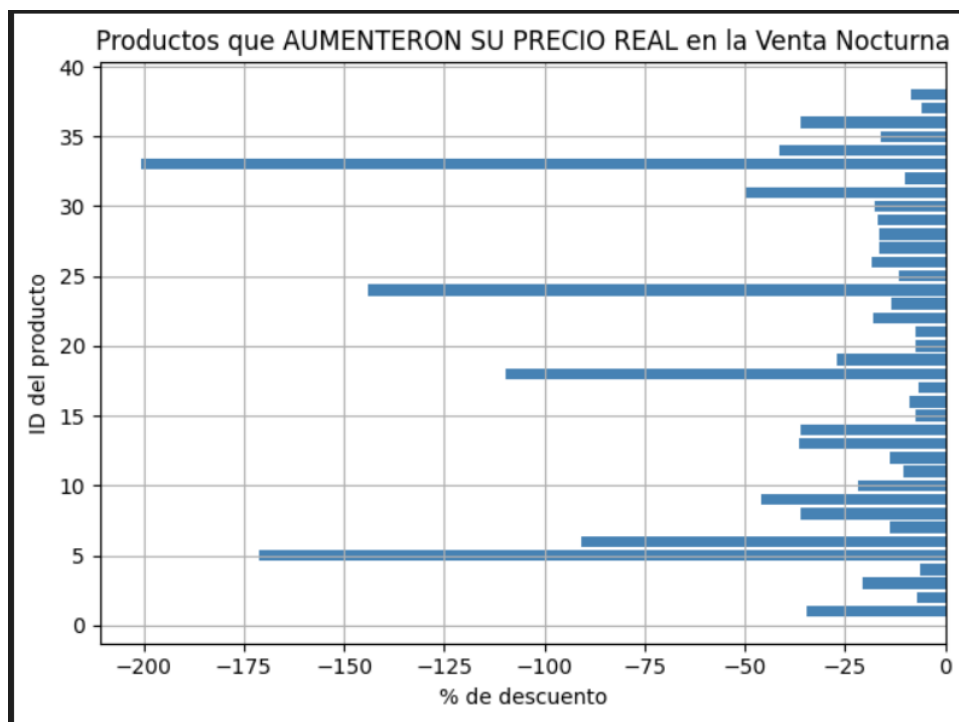
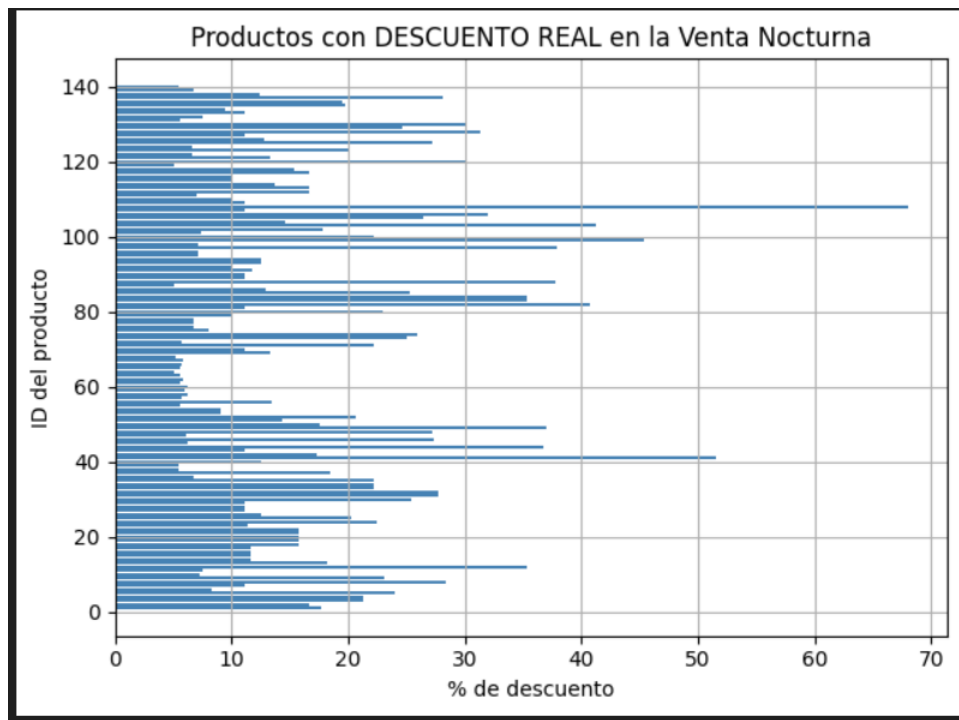
```
# TOP 15 BARATAS
plt.figure()
plt.bar(top_baratas1[numbers"numbersMarcanumbers"], top_baratas1[numbers"
numbersCon_numbersdescnumbers"], color=numbers"numberspinknumbers")
plt.title(numbers"numbersTopnumbers numbers15numbers numberssmarcasnumbers
numbersm snumbers numbersBARATASnumbers numbersennumbers numberselnumbers
numbersBLACKFRIDAYnumbers") #precio promedio con descuento
plt.xlabel(numbers"numbersMarcanumbers")
plt.ylabel(numbers"numbersPrecionumbers numberspromedionumbers numbers(numbersMXN
numbers)numbers")
plt.xticks(rotation=45, ha=numbers"numbersrightnumbers")
plt.tight_layout()
plt.show()

# TOP 15 CARAS
plt.figure()
plt.bar(top_caras1[numbers"numbersMarcanumbers"], top_caras1[numbers"
numbersCon_numbersdescnumbers"], color=numbers"numberspinknumbers")
plt.title(numbers"numbersTopnumbers numbers15numbers numberssmarcasnumbers
numbersm snumbers numbersCARASnumbers numbersennumbers numberselnumbers
numbersBLACKFRIDAYnumbers") # precio promedio con descuento
plt.xlabel(numbers"numbersMarcanumbers")
plt.ylabel(numbers"numbersPrecionumbers numberspromedionumbers numbers(numbersMXN
numbers)numbers")
plt.xticks(rotation=45, ha=numbers"numbersrightnumbers")
plt.tight_layout()
plt.show()
```

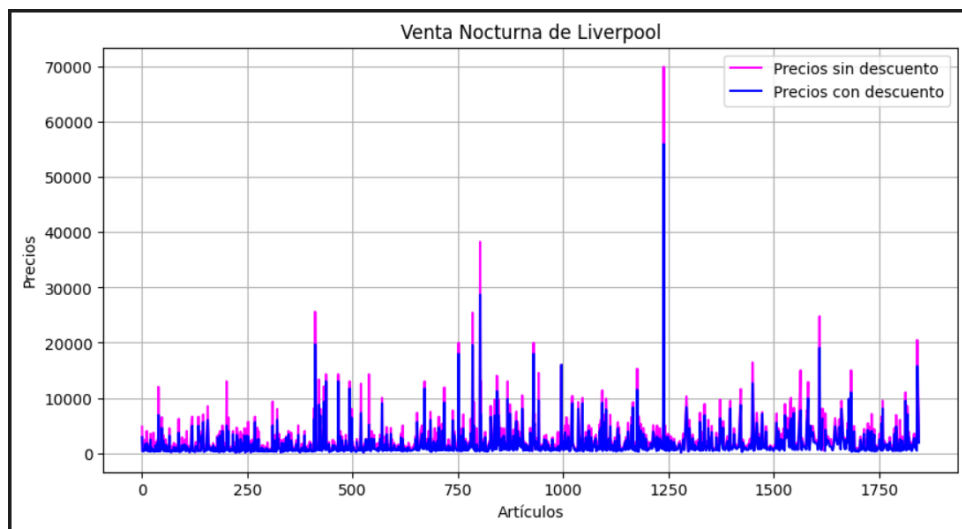
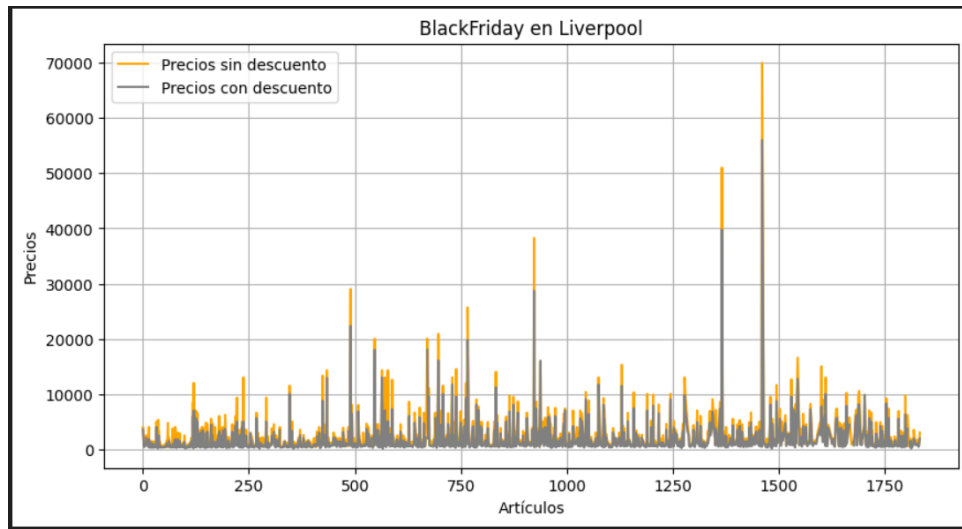
Resultados

El proceso de recolección y depuración de datos hizo evidente la importancia de utilizar filtros adecuados, controles para evitar duplicados y funciones de validación que garantizaran la calidad y coherencia de la información analizada. Del mismo modo, la integración de distintas bases de datos obtenidas en fechas diferentes permitió elaborar comparaciones directas entre los precios originales y los precios durante los eventos promocionales.

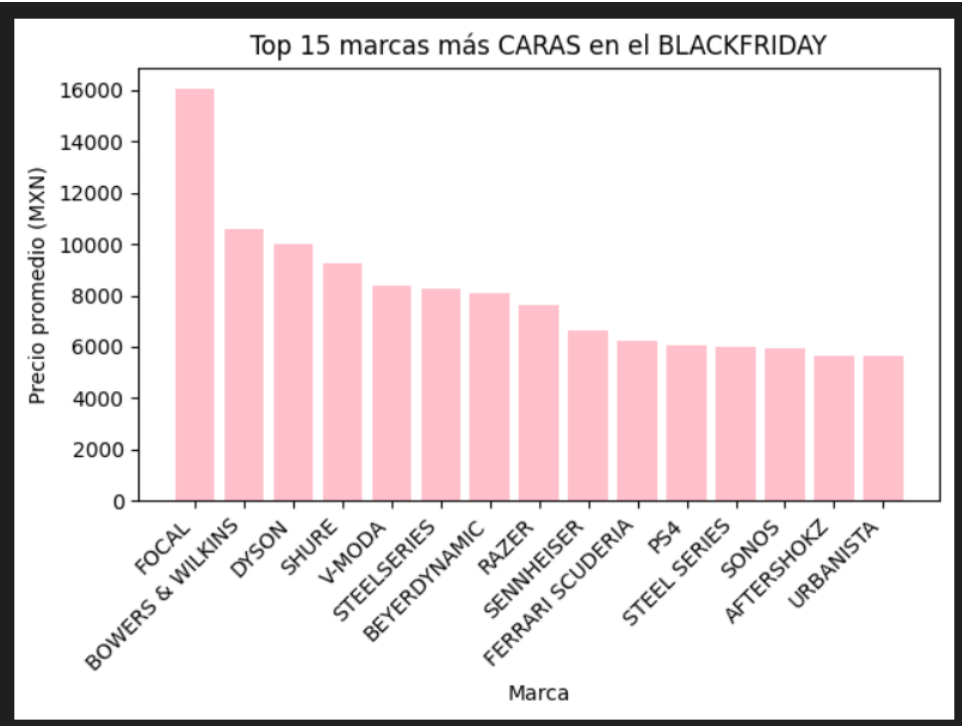
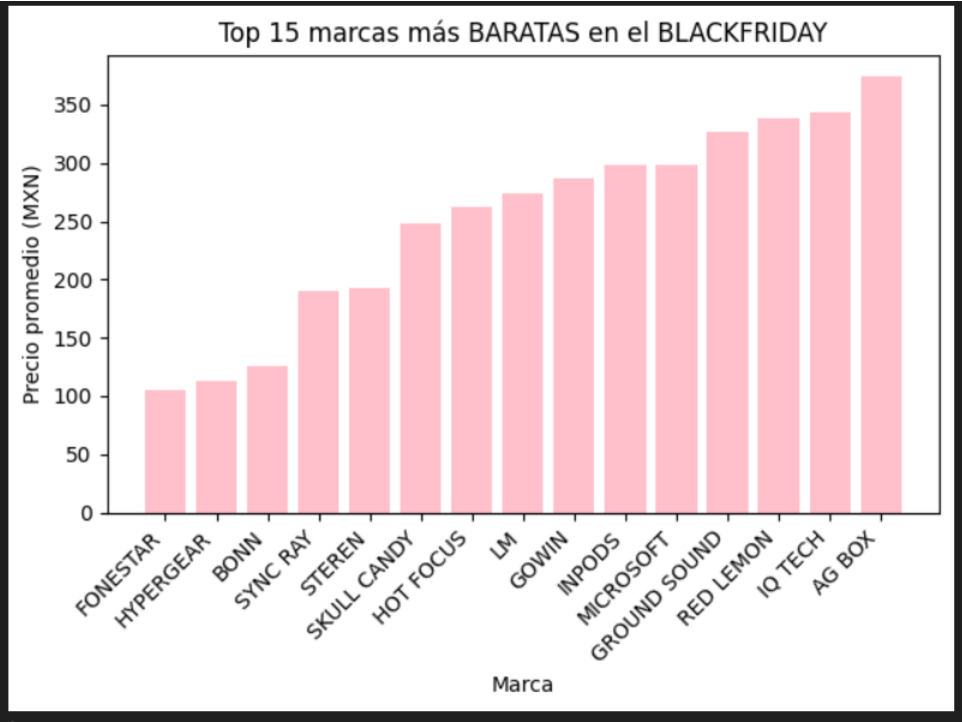


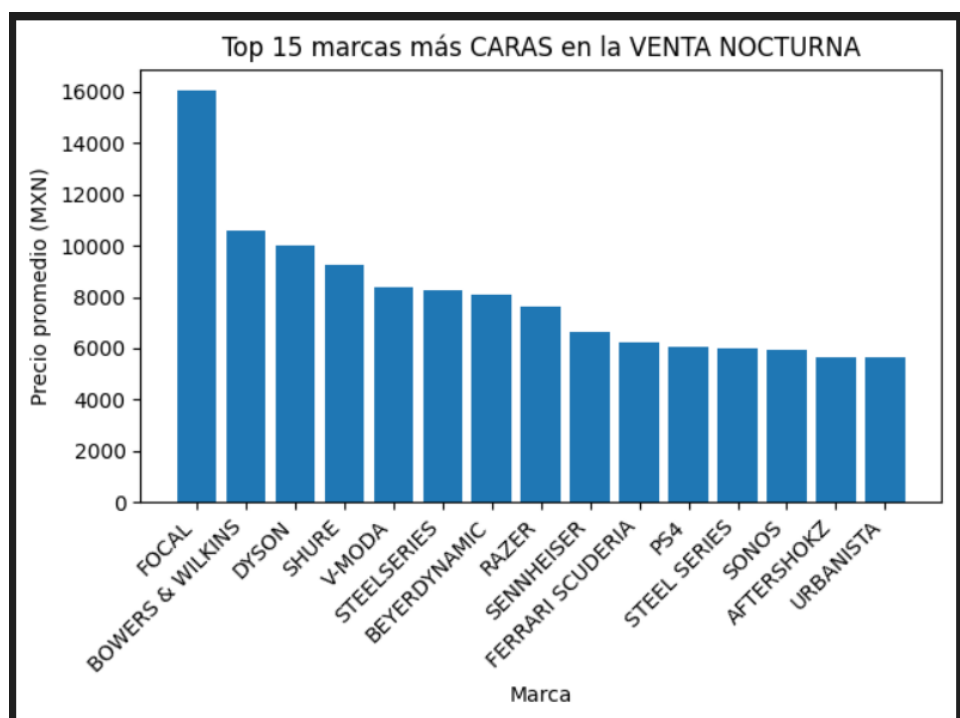
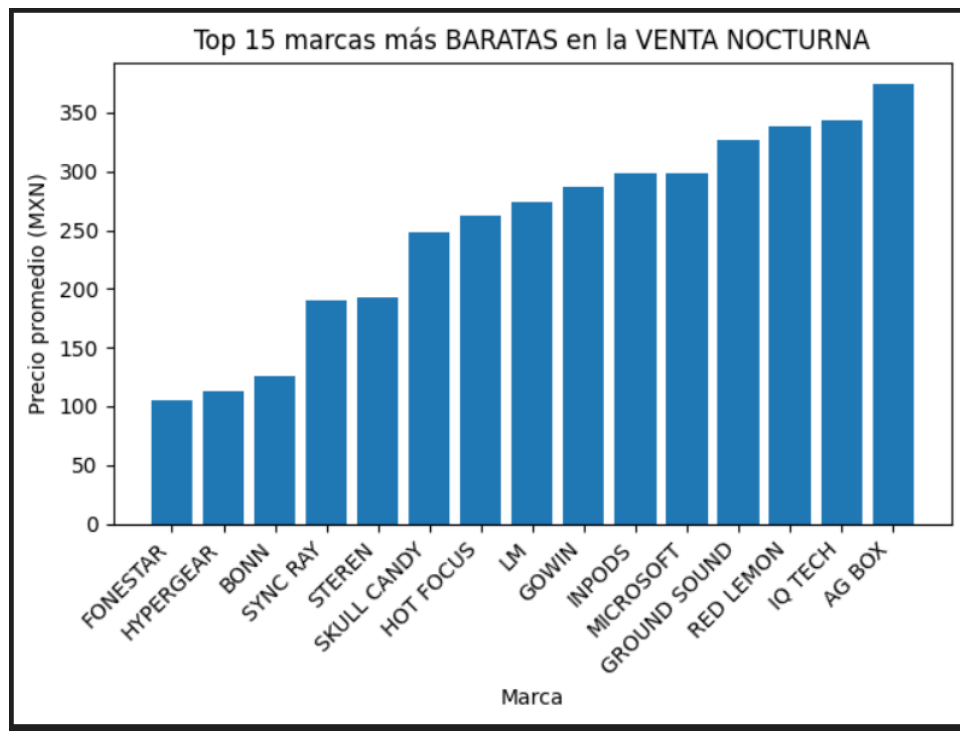


A partir de las comparaciones realizadas, se identificó que no todos los productos presentan descuentos reales, ya que una parte de ellos mantiene el mismo precio o incluso incrementa su costo durante las fechas de promoción. Asimismo, los análisis porcentuales evidenciaron casos de descuentos reales superiores al 5%, pero también aumentos significativos que sugieren prácticas de incremento previo de precios para simular rebajas. Esto confirma que los consumidores deben ser cautelosos al asumir que todos los precios promocionales representan un beneficio real.



La clasificación por marcas reveló diferencias importantes entre ellas, tanto en precios promedio como en la magnitud de los descuentos aplicados. Este tipo de análisis es especialmente útil para identificar qué marcas suelen ofrecer rebajas auténticas y cuáles muestran poca variación aun en periodos de oferta.





Finalmente, el proyecto también evidenció los retos técnicos inherentes al web scraping, tales como la compatibilidad con los controladores del navegador, actualizaciones automáticas, tiempos de carga variables y bloqueos de seguridad. No obstante, superar estas dificultades permitió obtener un sistema funcional y sólido para el análisis de precios en plataformas de comercio electrónico.

Complicaciones

Durante la elaboración del proyecto se presentaron diversas complicaciones relacionadas con el uso de Selenium para el proceso de web scraping. Una de las principales dificultades fue que no todos los entornos de ejecución o compiladores permiten abrir ventanas emergentes del navegador, lo cual es indispensable para que Selenium funcione correctamente. Por esta razón, el desarrollo del código se realizó en Visual Studio Code, empleando el lenguaje Python, que ofrece una mayor compatibilidad con los controladores de navegación.

Otra complicación importante fue la correcta instalación y configuración de todas las paqueterías y dependencias necesarias para ejecutar el proyecto. En particular, fue necesario instalar versiones específicas de Selenium, WebDriver Manager y los controladores del navegador (como ChromeDriver), asegurando que fueran compatibles tanto con la versión del navegador como con la versión de Python.

Además de lo anterior, se presentaron otras dificultades comunes en este tipo de proyectos, tales como:

- Actualizaciones automáticas del navegador, que provocaban incompatibilidad momentánea con Selenium o con el WebDriver.
- Bloqueos de seguridad o captchas en algunos sitios, que limitaban el scraping y obligaron a ajustar los tiempos de espera o utilizar técnicas como WebDriverWait.
- Tiempos de carga variables en las páginas web, que requerían la implementación de esperas explícitas y manejo de excepciones para evitar errores de sincronización.

Conclusiones

A partir del análisis realizado sobre los 1,802 audífonos disponibles en Liverpool durante tres periodos (antes del Black Friday, durante el Black Friday y durante la Venta Nocturna) se observa que los descuentos reales aplicados son mínimos. En ambas temporadas de ofertas, únicamente entre 120 y 140 productos presentan algún eel precio, lo que representa solo el 7.76

También se identificó que existe inflación previa de precios en ciertos productos. Aunque estos incrementos no son masivos, sí son relevantes: algunos artículos aumentan más del 50

Por otra parte, aunque existen casos donde los descuentos superan el 40-55

En general la publicidad de “ofertas” no se refleja necesariamente en reducciones significativas para la mayoría de los productos, pues un porcentaje reducido de artículos muestra descuentos verdaderos.

En varios casos, el precio se incrementa antes del evento para simular una rebaja.

Por ello, el Black Friday no garantiza precios más bajos en todo el catálogo, sino que algunos productos incluso resultan más caros durante la temporada.

Por eso es fundamental que los consumidores identifiquen qué marcas realmente ofrecen precios accesibles o descuentos genuinos, ya que la mayoría de las promociones analizadas presentan variaciones poco significativas o estrategias de precios poco transparentes.

En conclusión, el proyecto logró demostrar que el web scraping es una herramienta eficaz para evaluar la transparencia de precios en temporadas de descuentos, permitiendo comprobar, con datos reales, si las promociones ofrecidas corresponden a reducciones genuinas o a estrategias de mercado que podrían inducir a error al consumidor.